Lab 03: Detection Tuning & Rule Validation

An advanced exercise in understanding the recall / false-positive trade-off

Learning goals

- Run a narrow rule against a known, versioned set of synthetic cases.
- Explain how a rule can miss a process variant and trigger on legitimate activity.
- Make one controlled rule change and compare the resulting metrics.
- Write a compact detection specification that another learner can reproduce.

SCOPE This exercise uses synthetic CSV records and redacted placeholders. Do not run placeholder command lines. No malware, live scanning, or cloud resources are required.

Part A — Capture the baseline

From the website folder, run this command in PowerShell or a terminal:python public/labs/evaluate-detection-cases.py public/labs/detection-evaluation-cases.csv

Record the TP, FP, TN, FN counts and the metrics printed by the script. The script implements an intentionally narrow teaching rule: process name must be powershell.exe and the command line must include an -enc or -encodedcommand argument token.

BASELINE HYPOTHESIS The narrow rule should miss the synthetic pwsh.exe case, and it should flag the labelled benign encoded-maintenance case. That gives us both a false negative and a false positive to study.

Part B — Inspect the edge cases

Case	What happens in baseline	Question
case-007	powershell.exe with encoded marker; labelled benign; rule alerts	What context is needed before deciding whether this is malicious?
case-008	pwsh.exe with encoded marker; labelled suspicious; rule misses it	Which process-name assumption caused the false negative?
case-009	pwsh.exe with encoded marker; labelled benign; rule does not alert in baseline	What may happen to false positives if pwsh.exe is added to the rule?

Part C — Make one change

In a local copy of public/labs/evaluate-detection-cases.py, expand the process-name condition to include pwsh.exe as well as powershell.exe. Do not change any labels in the CSV. Run the same command again. Keep the original script so you can compare the baseline and revised behavior.

1. Confirm case-008 now alerts.
2. Confirm case-009 now alerts too, creating a false positive against its benign label.
3. Compare recall, precision, specificity, and F1 between the baseline and revised version.
4. Write down which metric improved, which worsened, and why.
5. Revert the change or save it as a separate version and record the rule version in your report.

Discussion: expected trade-off

Metric	Narrow baseline	With pwsh.exe added
TP / FP / TN / FN	2 / 1 / 4 / 1	3 / 2 / 3 / 0
Recall	0.667	1.000
Precision	0.667	0.600
Specificity	0.800	0.600
F1	0.667	0.750

These are the expected results for this deliberately constructed training corpus. They illustrate a rule trade-off, not the accuracy expected in a real environment. The CSV labels are manually assigned and the dataset is tiny.

Part D — Write a detection specification

- Rule ID and version.
- Purpose and observable behavior being detected.
- Required log source and exact fields.
- Detection logic and process-name normalization assumptions.
- Known false positives and likely blind spots.
- Test corpus version, counts, metrics, and exclusions.
- Recommended analyst checks and what cannot be concluded from the alert alone.

ADVANCED HABIT Do not optimize for a single metric in isolation. Select trade-offs using the use case, log quality, investigation capacity, and a representative evaluation set.

Further reading

- Sigma rule repository
- Sigma specification
- MITRE ATT&CK

A real lab for these notes

Topic: Tune detection rules, compare false positives, and validate on a fixed sample. These pages connect the existing OpenCyberPath notes to specific provider-hosted exercises. The card label is intentionally generic; the exact task name and direct URL are listed here so you can open the right exercise instead of a whole catalogue.

Concept checkpoints

- Change one condition at a time and keep a record of the baseline.
- A narrow process-name match can miss variants such as pwsh.exe.
- Broadening a rule changes the false-positive/false-negative trade-off.
- YARA rules should first be tested against intentionally provided sample fixtures, never unknown files.

Yara Wizards *[CyberDefenders]*
https://cyberdefenders.org/blueteam-ctf-challenges/yara-wizards/
Access note: Direct detection-engineering challenge page. Sign-in, downloadable sample files, and current access tier can vary; inspect the page before starting.

Sigma 101 *[CyberDefenders]*
https://cyberdefenders.org/blueteam-ctf-challenges/sigma-101/
Access note: Direct challenge page for Sigma rules; check whether the current challenge is downloadable and what tools are required.

ACCESS CHECK Open the exact task page and verify the current login, account tier, VM launch button, free/paid terms, and any cost notes before starting. A provider may change availability or retire a challenge.

Before you click Start

- Write down the provider's target/scope and use only the instance it creates for you.
- Keep this notebook beside the exercise. Use its original course data before trying commands on a local machine.
- Capture timestamps, filenames, row/packet IDs, and a short explanation; redact credentials, tokens, and personal data.
- If a page says retired, premium, or downloadable-only, use the alternative listed in the access note or return to the matching interactive lab above.

Commands you can explain

Only test rules against the packaged training rows or files provided by the named challenge. No malware is required for this notebook.

First download the matching sample file(s) from the existing card when a command names one. Keep the data in a practice folder and never execute strings stored in evidence.

PYTHON / BASELINE RUN

```
python .\evaluate-detection-cases.py .\detection-evaluation-cases.csv
```

Save the original result before changing the detection logic.

POWERSHELL / INSPECT PROCESS VARIANTS

```
Import-Csv .\detection-evaluation-cases.csv | Group-Object process | Select-Object Name,Count
```

Find which process names are included in the test set.

POWERSHELL / REVIEW FALSE POSITIVES

```
Import-Csv .\detection-evaluation-cases.csv | Where-Object {$_.activity_label -eq 'benign'} | Format-
Table case_id,process,command_line -AutoSize
```

Read benign cases that a broad rule might match. Do not execute command_line values.

YARA / INSPECT INSTALLED TOOL

```
yara --version
```

Confirm YARA is installed before using the provider's sample set.

YARA / SCAN ONLY SUPPLIED FIXTURES

```
yara .\training-marker.yar .\training-fixtures\
```

Example pattern only: use the rule and directory names supplied by the assigned lab; never aim a test rule at arbitrary system directories.

COMMAND HABIT Check the current working directory, shell, target, and expected effect before pressing Enter. Save the exact command and the output that mattered.

Guided lab run

- 1. Open the direct YARA or Sigma challenge and read the required tool list.
- 2. Record the rule's original purpose, log source, fields, and expected activity before editing it.
- 3. Run the baseline evaluator against the unchanged sample CSV.
- 4. Make one controlled change (for example, add a known benign exclusion or document a missing process variant), then re-run.
- 5. Compare each confusion-matrix count and include a short rationale for any increase in false positives or false negatives.

Evidence checklist

- [ ] What exact lab instance or sample file was used?
- [ ] Which command/query and time range produced the observation?
- [ ] What is the directly observed fact, separate from your inference?
- [ ] What benign/alternative explanation could fit?
- [ ] What extra record or control would help confirm the conclusion?

Check your understanding

Q Why is a single-change experiment easier to explain?

Think it through: It makes the effect of that change easier to attribute and reproduce.

Q Can YARA prove a file is malicious from one string match?

Think it through: No. A match is a clue; inspect file context, source, signatures, hashes, and other evidence.

Q What needs to accompany the final tuned rule?

Think it through: Log source, required fields, test corpus version, metrics, exclusions, false-positive notes, and limitations.

NOTEBOOK CLOSE-OUT Finish with: Observation / Evidence source / Why it matters / Alternative explanation / Next check / Limitation. A careful "not enough evidence yet" is a valid professional result.